

SECTION 3 OPERAND DATA FORMATS

The following paragraphs describe the MC68881/MC68882 (FPCP) operand data formats. Seven data formats are supported: three signed binary integer formats, three binary floating-point formats, and one packed binary-coded decimal (BCD) floating-point format. All data formats are supported uniformly by all arithmetic and transcendental instructions. These formats are as follows:

- Byte Integer (B)
- Word Integer (W)
- Long Word Integer (L)
- Single Precision Real (S)
- Double Precision Real (D)
- Extended Precision Real (X)
- Packed Decimal Real (P)

The capital letter in parentheses is the suffix added to an instruction in the assembly language syntax to specify the data format of operands external to the FPCP. All data formats are organized in memory consistently with the M68000 Family data organization, i.e., the most significant byte is located at the lowest address (nearest \$00000000), with each successively less significant byte located at the next address (N + 1, N + 2, etc.). The least significant byte is located at the highest address (nearest \$FFFFFFF).

Within the floating-point data formats, there are five types of numbers that can be represented: normalized numbers, denormalized numbers, zeros, infinities, and not-a-numbers (NaNs). These data types are represented with special encodings corresponding to each data format.

3.1 INTEGER DATA FORMATS

The three signed (twos complement) integer data formats supported by the FPCP (byte, word, and long word) are identical to those supported by the M68000 Family architecture (see Figure 3-1).

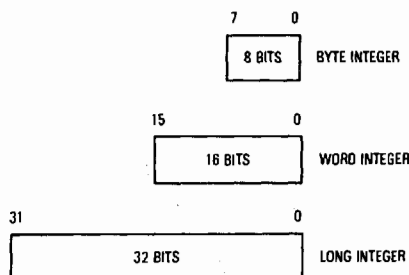


Figure 3-1. Signed Integer Data Formats



all FPCP internal operations are performed in full extended precision format, signed or unsigned. All operands are converted to extended precision values before the specified operation is performed. Thus, mixed mode arithmetic is implicitly supported.

3.2 BINARY REAL DATA FORMATS

3

Floating-point numbers can be encoded in any of three binary real data formats: single precision (32 bits), double precision (64 bits), and double-extended precision (96 bits, 80 of which are used). All three of these formats fully comply with the *IEEE Standard for Binary Floating-Point Arithmetic*.

NOTE

The single-extended precision data format defined in the IEEE standard is redundant in a device that supports the double-extended precision format. Thus, all references in this manual to extended precision imply double-extended precision as defined by the IEEE standard.

Since all FPCP internal operations are performed in extended precision, single and double precision operands are converted to extended precision values before the specified operation is performed. Thus, mixed mode arithmetic is implicitly supported. The memory formats for the real data formats are shown in Figure 3-2.

The exponent in all three binary formats is an unsigned binary integer with an implied bias added to it. The bias values for single, double, and extended precision are 127, 1023, and 16383, respectively. When the bias is subtracted from the value of the exponent, the result represents a signed twos-complement power of two that yields the magnitude of a normalized floating-point number when multiplied by the mantissa. Since biased exponents are used, a program can execute an integer compare instruction (CMP) to compare floating-point numbers in memory (regardless of the absolute magnitude of the exponents).

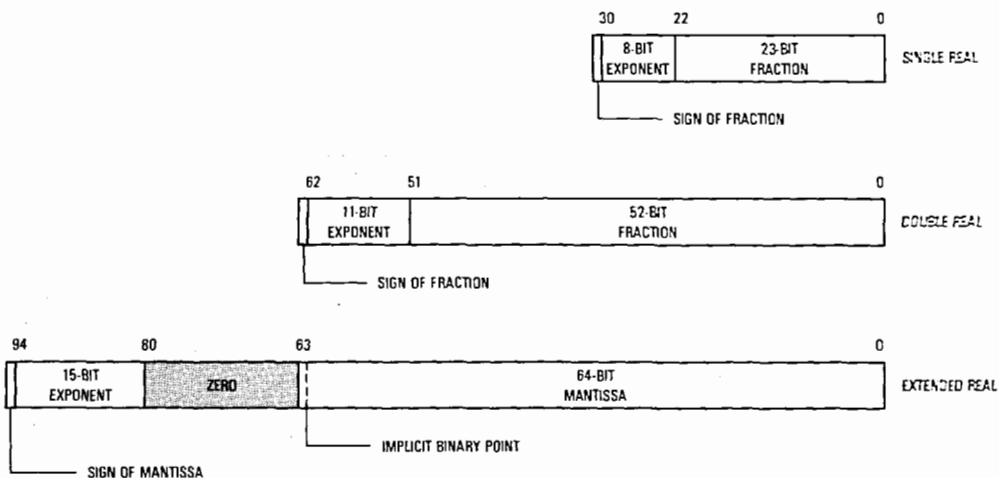


Figure 3-2. Binary Real Data Formats

Formats for single and double precision numbers differ slightly from the data formats intended precision numbers in the representation of the mantissa. A normalized mantissa, for all three precisions, is always in the range [1.0 . . . 2.0). The extended precision data format explicitly represents the entire mantissa, including the explicit integer part bit. However, for single and double precision data formats, only the fractional portion of the mantissa is explicitly represented and the integer part, always one, is implied.

The IEEE standard has created the term “significand” to bridge this difference and to avoid the historical implications of the term mantissa. The IEEE standard defines a significand as the component of a binary floating-point number that consists of an explicit or implicit leading bit to the left of the implied binary point. This manual uses the terms mantissa and significand, defined as follows, interchangeably.

Single Precision Mantissa = Single Precision Significand
= 1.<23-Bit Fraction Field>
Double Precision Mantissa = Double Precision Significand
= 1.<52-Bit Fraction Field>
Extended Precision Mantissa = Extended Precision Significand
= 1.Fraction
= <64-Bit Mantissa Field>

NOTE

Throughout this manual, ranges are specified using traditional set notation with the format “bound . . . bound” specifying the boundaries of the range. The type of brackets enclosing the range defines whether the endpoint is inclusive or exclusive. A square bracket indicates inclusive, and a parenthesis indicates exclusive. For example, the range specification “[1.0..2.0]” defines the range of numbers greater than or equal to 1.0 and less than or equal to 2.0. The range specification “(0.0... + inf]” defines the range of numbers greater than 0.0 and less than or equal to positive infinity.

Each of the three floating-point data formats can represent five unique floating-point data types:

- Normalized Numbers
- Denormalized Numbers
- Zeros
- Infinities
- Not-A-Numbers (NaNs)

The normalized data type never uses the maximum or minimum exponent value for a given format (except for the extended precision format, see following note). These exponent values in each precision are reserved for representing the special data types: zeros, infinities, denormalized numbers, and NaNs. Details of each type of number for each format are shown in **3.6 DATA FORMAT DETAILS**.

NOTE

There is a subtle difference between the definition of an extended precision number with an exponent equal to zero and a single or double precision number with an exponent equal to zero. If the exponent of a single or double precision number is zero, the number is defined to be denormalized, and the implied integer bit is also a zero. However, an extended precision number with an exponent of zero may have an explicit integer bit equal to one, which results in a normalized number (even though the exponent is equal to the minimum value).

mplicity, the following discussion treats all three real formats in the same manner, an exponent value of zero identifies a denormalized number. However, it should be noted that the extended precision format may deviate from this rule.

3.2.1 Normalized Numbers

3

Normalized numbers encompass all representable real values between the overflow and underflow thresholds, i.e., those numbers whose exponents lie between the maximum and minimum values. Normalized numbers may be positive or negative. For normalized numbers, the implied integer part bit in single and double precision is a one (1). In extended precision, the integer bit is explicitly a one (1). See Figure 3-3.

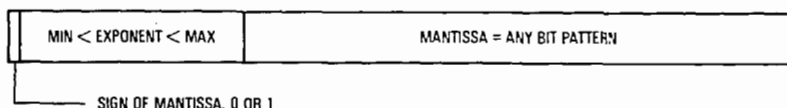


Figure 3-3. Format of Normalized Numbers

3.2.2 Denormalized Numbers

Denormalized numbers represent real values near the underflow threshold (underflow is detected for a given data format and operation when the result exponent is less than or equal to the minimum exponent value). Denormalized numbers may be positive or negative. For denormalized numbers, the implied integer part bit in single and double precision is a zero (0). In extended precision, the integer bit is explicitly a zero (0). See Figure 3-4.

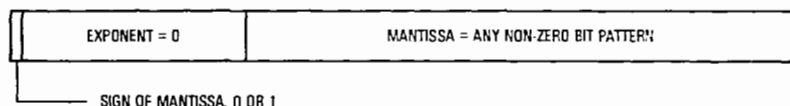


Figure 3-4. Format of Denormalized Numbers

Traditionally, floating-point number systems perform a "flush-to-zero" when underflow is detected. This leaves a large gap in the number line between the smallest magnitude normalized number and zero. The IEEE standard implements gradual underflows: the result mantissa is shifted right (denormalized) while the result exponent is incremented until the result exponent reaches the minimum value. If all mantissa bits of the result are shifted off to the right during this denormalization, the result becomes zero. In many instances, gradual underflow limits the potential underflow damage to no more than a round-off error. (This underflow and denormalization description ignores the effects of rounding and the user selectable rounding modes.) Thus, the large gap in the number line created by "flush-to-zero" floating-point number systems is filled with representable (denormalized) numbers in the IEEE "gradual underflow" floating-point number system.

Since the extended precision data format has an explicit integer part bit, a number can be formatted with a nonzero exponent (less than the maximum value) and a zero integer bit, which is not defined by the IEEE standard. Such a number is called an unnormalized number. The MC68881 never generates an unnormalized number as the result of any operation. Unnormalized inputs are always converted to normalized or denormalized numbers or zero before being used. Thus, as required by the IEEE standard, the FPCP does not distinguish between redundant encodings of extended precision values.

3.2.3 Zeros

Zeros are signed (positive or negative) and represent the real values $+0.0$ and -0.0 . See Figure 3-5.

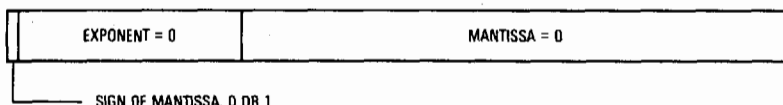
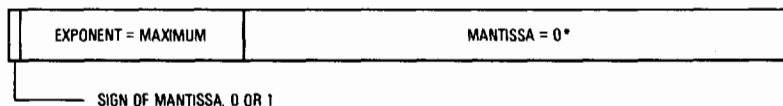


Figure 3-5. Format of Zero

3.2.4 Infinities

Infinities are signed (positive or negative) and represent real values that exceed the overflow threshold. Overflow is detected for a given data format and operation when the result exponent is greater than or equal to the maximum exponent value. (This overflow description ignores the effects of rounding and the user selectable rounding modes.) See Figure 3-6. For extended precision infinities, the most significant bit of the mantissa (the integer bit) can be either one or zero.



*For the extended precision format, the most significant bit of the mantissa (the integer bit) is a don't care.

Figure 3-6. Format of Infinity

3.2.5 Not-A-Numbers

When created by the FPCP, not-a-numbers (NaNs) represent the results of operations that have no mathematical interpretation, such as infinity divided by infinity. All operations involving a NaN operand as an input return a NaN result. When created by the user, NaNs can protect against uninitialized variables and arrays, or represent user-defined special



Figure 3-7. Format of Not-A-Numbers

Two different types of NaNs are implemented by the FPCP. The value of the most significant bit (MSB) of the fraction identifies the type. The identifying bit is the MSB of the mantissa for single and double precision and the MSB of the mantissa minus one for extended precision. NaNs with a leading fraction bit equal to one are nonsignaling NaNs; NaNs with a leading fraction bit equal to zero are signaling NaNs (SNANs). A SNAN can be used as an escape mechanism for a user-defined non-IEEE data type. The FPCP never creates a SNAN as a result of an operation.

The IEEE specification defines the manner in which a NaN is processed when used as an input to an operation. Particularly, if a SNAN is used as an input and the SNAN trap is not enabled, a nonsignaling NaN must be returned as the result. The FPCP accomplishes this by using the source SNAN, setting the most significant bit of the fraction, and storing the resultant nonsignaling NaN in the destination. Due to the IEEE formats for NaNs, the result of setting the most significant fraction bit of a SNAN is always a nonsignaling NaN.

When NaNs are created by the FPCP, the NaNs always contain the same bit pattern in the mantissa; for any precision, all bits of the mantissa are ones. When a NaN is created by the user, any nonzero bit pattern can be stored in the mantissa.

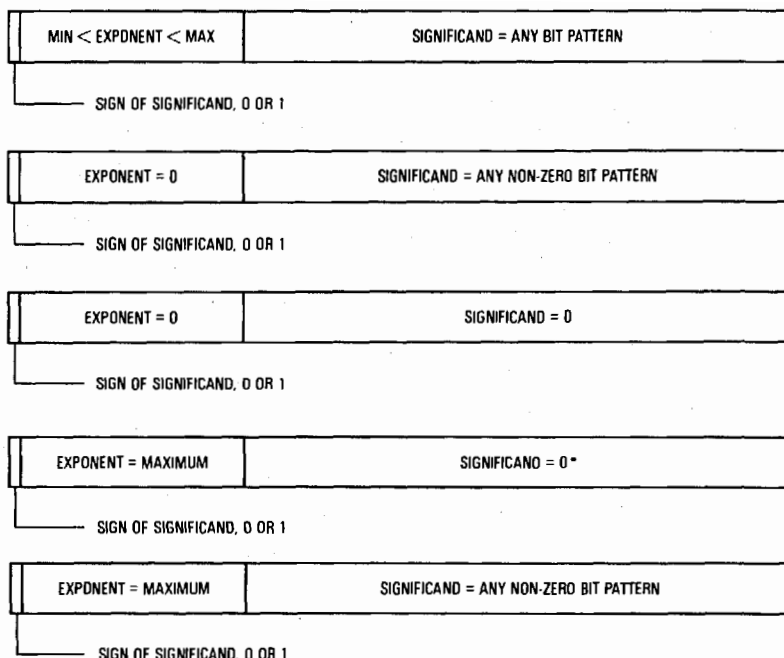
3.2.6 Binary Real Data Summary

Figure 3-8 presents a summary, for quick reference, of the five floating-point data types for the single, double, and extended precision formats.

3.3 PACKED DECIMAL REAL DATA FORMAT

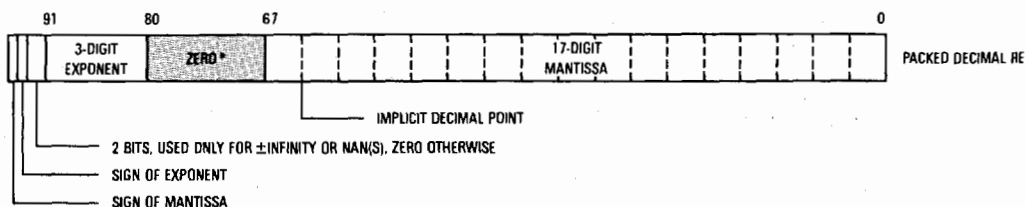
The packed decimal floating-point data format consists of a 24 digit packed decimal string as shown in Figure 3-9. A decimal floating-point source operand is converted to an extended precision value before the specified operation is performed. Thus, mixed mode arithmetic is implicitly supported.

The packed decimal representation for the special data types of zero, infinity, and NaN is described in **3.6 DATA FORMAT DETAILS**, which also defines all possible data patterns in the packed decimal data format.



*For the extended precision format, the most significant bit of the significand (the integer bit) is a don't care.

Figure 3-8. Binary Real Data Type Summary



*Unless a binary-to-decimal conversion overflow occurs

Figure 3-9. Packed Decimal Real Data Format

3.4 INTERNAL DATA FORMAT

All FPCP internal operations are performed in extended precision. All external operands, regardless of data format, are converted to extended precision values before the specified operation is performed.

The format used in the eight floating-point data registers is identical to the extended precision data format described previously and in **3.6 DATA FORMAT DETAILS** (with the

The format of an intermediate result is shown in Figure 3-10. The intermediate result exponent for some dyadic operations (multiply and divide) can easily overflow or underflow the 15-bit exponent. In order to simplify overflow and underflow detection, intermediate results in the FPCP maintain a 17-bit two's-complement integer exponent. When an overflow or underflow intermediate result is detected, the intermediate 17-bit exponent is always converted into a 15-bit biased exponent before it is stored in a floating-point data register. Additionally, the mantissa is maintained internally as 67 bits for rounding purposes, but is always rounded to 64 bits (or less, depending on the selected rounding precision) before it is stored in a floating-point data register.

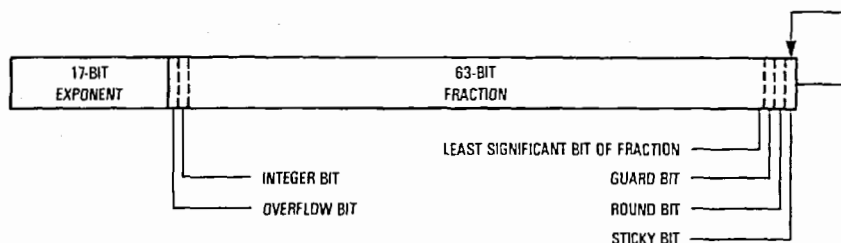


Figure 3-10. Intermediate Result Format

3.5 FORMAT CONVERSIONS

Two cases of conversion between two data formats are:

- Converting an operand in any memory data format to the extended precision data format and storing it in a floating-point data register, or using it as the source operand for an arithmetic operation.
- Converting the extended precision value in a floating-point data register to any data format and storing it in a memory destination.

3.5.1 Conversion to Extended Precision Data Format

Since the internal data format used by the FPCP is always extended precision, all external operands, regardless of data format, are converted to extended precision values before the specified operation is performed. If the external operand, regardless of data format, is a denormalized number, the number is normalized before the specified operation is performed. Conversion and normalization apply not only to loading a floating-point data register but also to external operands involved in arithmetic operations.

Since floating-point data registers always contain extended precision data format values, an external extended precision denormalized number moved into a floating-point data register is stored as an extended precision denormalized number. In this case, the number



normalized and then denormalized before it is stored in the designated floating-point data register. This method simplifies the handling of all other data formats and types.

If an external operand is an extended precision unnormalized number, the number is normalized before it is used in an arithmetic operation. If the external operand is an extended precision unnormalized zero (i.e., with a mantissa of all zeros), the number is converted to an extended precision normalized zero before the specified operation is performed. This normalization and conversion applies not only to external unnormalized operands involved in arithmetic operations, but also applies to loading a floating-point data register. Note that the regular use of unnormalized inputs defeats the purpose of the IEEE standard and may produce gross inaccuracy in the results.

3.5.2 Conversions to Other Data Formats

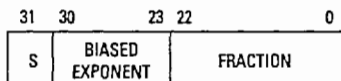
Conversion from the extended precision data format to any of the other six data formats occurs when the contents of an FPCP floating-point data register are stored to memory or an MPU data register. Since no operation performed by the FPCP can create an unnormalized result, the result of moving a floating-point data register to an extended precision external destination can never be an unnormalized number.

3.6 DATA FORMAT DETAILS

The following paragraphs provide the format specification details for the single (S), double (D), extended (X) precision binary real, and packed decimal (P) real string data formats. Refer to Tables 3-1 through 3-4 and Figure 3-11.

Table 3-1. Single Precision Binary Real Format

memory Format:



Field Size (in Bits):

s = Sign	1
e = Biased Exponent	8
f = Fraction	23
Total	32

Interpretation of Sign:

Positive Mantissa, s =	0
Negative Mantissa, s =	1

Normalized Numbers:

Bias of e	+ 127 (\$7F)
Range of e	$0 < e < 255$ (\$FF)
Range of f	Zero or Nonzero
Mantissa = Significand =	1.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^e - 127 \times 1.f$

Denormalized Numbers:

e = Format Minimum =	0 (\$00)
Bias of e	+ 126 (\$7E)
Range of f	Nonzero
Mantissa = Significand =	0.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{-126} \times 0.f$

Signed Zeros:

e = Format Minimum =	0 (\$00)
f = Mantissa = Significand =	0.f = 0.0

Signed Infinities:

e = Format Maximum =	255 (\$FF)
f = Mantissa = Significand =	0.f = 0.0

NANs (Not-A-Number):

s =	Don't Care
e = Format Maximum =	255 (\$FF)
f =	Non-Zero
Representation of f	0.1xxxx...xxxx, Nonsignaling
	0.0xxxx...xxxx, Signaling
	Nonzero Bit Pattern
	.11111...1111

xxxx...xxxx

f When Created by the FPCP

Ranges (Approximate):

Maximum Positive Normalized	3.4×10^{38}
Minimum Positive Normalized	1.2×10^{-38}
Minimum Positive Denormalized	1.4×10^{-45}

Table 3-2. Double Precision Binary Real Format
Memory Format:

63	62	52	51	0
S	BIASED EXPONENT		FRACTION	

Field Size (in Bits):

s = Sign	1
e = Biased Exponent	11
f = Fraction	52
Total	64

Interpretation of Sign:

Positive Mantissa, s =	0
Negative Mantissa, s =	1

Normalized Numbers:

Bias of e	+ 1023
Range of e	$0 < e < 2047$ (\$7FF)
Range of f	Zero or Nonzero
Mantissa = Significand =	1.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{e-1023} \times 1.f$

Denormalized Numbers:

e = Format Minimum =	0 (\$000)
Bias of e	+ 1022 (\$3FE)
Range of f	Nonzero
Mantissa = Significand =	0.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{-1022} \times 0.f$

Signed Zeros:

e = Format Minimum =	0 (\$00)
f = Mantissa = Significand =	0.f = 0.0

Signed Infinities:

e = Format Maximum =	2047 (\$7FF)
f = Mantissa = Significand =	0.f = 0.0

NANs (Not-A-Number):

s =	Don't Care
e = Format Maximum =	2047 (\$7FF)
f =	Nonzero
Representation of f	0.1xxxx...xxxx, Nonsignaling
	0.0xxxx...xxxx, Signaling
xxxx...xxxx	Nonzero Bit Pattern
f When Created by the FPCP	.1111...1111

Ranges (Approximate):

Maximum Positive Normalized	18×10^{307}
Minimum Positive Normalized	2.2×10^{-308}
Minimum Positive Denormalized	4.9×10^{-324}

Memory Format:

95	94	80	79	64	62	0
S	BIASED EXPONENT	ZERO	INTEGER PART FRACTION			

Field Size (in Bits):

s = Sign	1
e = Biased Exponent	15
u = Zero, Reserved	16
j = Integer Part	1
f = Fraction	63
Total	96

Interpretation of Unused Bits:

Input	Don't Care
Output	All Zeros

Interpretation of Sign:

Positive Mantissa, s =	0
Negative Mantissa, s =	1

Normalized Numbers:

Bias of e	+16383 (\$3FFF)
Range of e	$0 \leq e < 32767$ (\$7FFF)
j =	1
Range of f	Zero or Nonzero
j.f = Mantissa = Significand =	1.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^e - 16383 \times j.f$

Denormalized Numbers:

e = Format Minimum	0 (\$0000)
Bias of e	+16383 (\$3FFF)
j =	0
Range of f	Nonzero
j.f = Mantissa = Significand =	0.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{-16383} \times 0.f$

Signed Zeros:

e = Format Minimum =	0 (\$0000)
j.f = Mantissa = Significand =	0.0

Signed Infinities:

e = Format Maximum =	32767 (\$7FFF)
j =	Don't Care
j.f = Mantissa = Significand	j.000...0000

NANs (Not-A-Numbers):

s =	Don't Care
j =	Don't Care
e = Format Maximum =	32767 (\$7FFF)
f =	Nonzero
Representation of f	j.1xxx...xxxx, Non-Signaling
	j.0xxx...xxxx, Signaling
xxx...xxxx	Non-Zero Bit Pattern
f When Created by the FPCP	1.1111...1111

Ranges (Approximate):

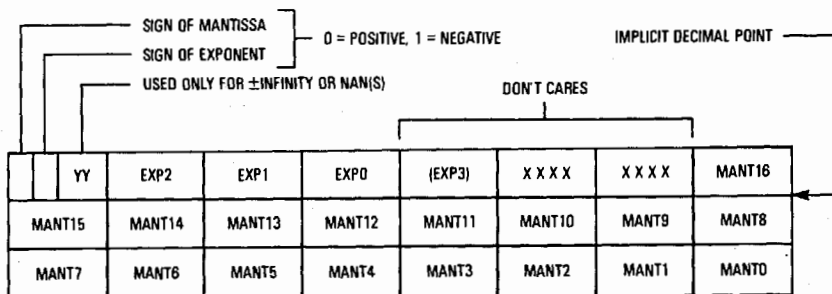
Maximum Positive Normalized	6×10^{4931}
Minimum Positive Normalized	8×10^{-4933}
Minimum Positive Denormalized	9×10^{4952}

Table 3-4. Decimal String Type Definitions

Operand Type	Word 5					Word 4	Words 3-0
	15	14	13	12	11...0	15...0	16-Digit Fraction
	SM	SE	y	y	3-Digit Exponent	1-Digit Integer	
± INFINITY	0/1	1	1	1	FFFF	xxx	\$00...00
± NAN	0/1	1	1	1	FFF	xxxx	Nonzero (see Note 1)
± SNAN	0/1	1	1	1	FFF	xxxx	Nonzero (see Note 1)
± ZERO	0	0/1	x	x	\$000-\$999	xxx0	\$00...00
- ZERO	1	0/1	x	x	\$000-\$999	xxx0	\$00...00
+ In-Range	0	0/1	x	x	\$000-\$999	xxx0-\$xxx9	\$00...01-\$99...\$99
- In-Range	1	0/1	x	x	\$000-\$999	xxx0-\$xxx9	\$00...01-\$99...\$99

NOTES:

1. A decimal string with the SE and y bits set, an exponent of FFFF, and a nonzero 16-digit decimal fraction is a NAN. When this string is used by the FPCP, the fraction part of the NAN is moved bit-for-bit into the extended precision mantissa of a floating-point register. The exponent of the register is set to signify a NAN, but no decimal-to-binary conversion or any other conversion is performed. Therefore, the most significant bit of the most significant digit in the decimal fraction (most significant bit of MANT15) is a don't care (as in extended NANs), and the most significant bit minus one of MANT15 is the signaling NAN (SNAN) bit. If the NAN bit is a zero, then it is a SNAN.
2. If a nondecimal digit [\$A...\$F] appears in the exponent of a zero, the number is converted to a true zero. The FPCP does not detect nondecimal digits [\$A...\$F] in the exponent, integer, or fraction digits of an in-range decimal string. These nondecimal digits are converted to binary in the same manner as decimal digits; however, the result is probably useless although it is repeatable.
3. Since in-range numbers cannot overflow or underflow when converted to extended precision, normalized extended precision numbers are always produced by conversion from the decimal data format.



MANTn Is the nth digit of the mantissa.
 EXPn Is the nth digit of the exponent. EXP3 is only generated during a move out operation if the source operand exponent exceeds the magnitude of a three digit exponent; otherwise, it is a don't care. Only EXP0-EXP2 are used for input.
 XXXX Are don't care bits, which are zero when written and ignored when read.

Figure 3-11. Packed Decimal Real Data Format Detail

